

Тестовое задание: Mini Booking (Flutter + React + Node.js GraphQL)

****Вакансия:**** Flutter / Fullstack-разработчик (Middle)

****Цель:**** показать умение быстро собрать рабочий вертикальный срез на всем стеке:

****backend (Node.js + Apollo GraphQL)**** → ****Flutter (mobile + Windows)**** → ****web (React/TS)****.

****Формат сдачи:**** репозиторий + Docker + видео-демо.

1) Описание задачи

Сделать мини-систему бронирования:

- есть ****2 отеля****
- у каждого отеля есть ****номера****
- у номеров есть ****занятость по датам****
- можно:

- 1) посмотреть доступность номера на диапазон дат
- 2) ****забронировать**** номер на диапазон дат
- 3) ****отменить**** бронь

****Ограничение:**** брони ****не должны пересекаться**** для одного номера (допускается только одна активная бронь на конкретный период).

> Структуру сущностей/поля и внутреннюю модель данных определяете сами. Важно, чтобы сценарии работали и были воспроизводимы.

2) Backend (обязательно)

****Stack:**** Node.js + Apollo GraphQL (TypeScript — плюс)

Требования

- GraphQL API должно покрывать:
 - получение списка отелей и номеров
 - получение занятости/доступности номера на диапазон дат
 - создание брони
 - отмену брони
- Хранилище Postgres
- ****Seed данных при старте****:
 - 2 отеля
 - несколько номеров в каждом
 - несколько броней для демонстрации конфликтов
- Обработка ошибок и валидация диапазона дат
- Логирование ключевых операций (создание/отмена)

3) Flutter (обязательно)

3.1 Mobile (iOS/Android)

Минимум:

- Навигация: ****Отели → Номера → Номер****
- В карточке номера:

- выбор диапазона дат
- проверка доступности
- бронирование
- отмена брони (из списка броней номера или из текущей активной брони)
- Интеграция с GraphQL (типобезопасный клиент — плюс, но не обязательно)

3.2 Windows-виджет (Flutter desktop) — light

Сделать мини-приложение/виджет (можно отдельным таргетом), который показывает:

- 2 отеля + краткий статус по номерам (например: “свободно/занято сегодня” или “ближайшая бронь”)
- кнопку ****Refresh/Обновить****
- минимум один экран деталей (или раскрытие информации) — на ваше усмотрение

4) Web (обязательно)

****Stack:**** React + TypeScript + Apollo Client

Минимум:

- список отелей/номеров
- просмотр броней номера
- создание/отмена брони (через тот же GraphQL)

UI можно простой: важнее структура, читаемость и работоспособность.

5) Docker (обязательно)

Нужно приложить:

- ****docker-compose.yml****, который поднимает backend + БД (если используется) одной командой
- инструкция в README:
 - как запустить: ``docker compose up --build``
 - какие порты и URL
 - как открыть GraphQL endpoint / playground (если есть)

6) Видео-демо (обязательно)

Одно видео ****3–7 минут****, где видно:

- 1) запуск проекта (или уже поднят, но показываете команды/compose)
- 2) работа backend (GraphQL playground/Insomnia/Postman — кратко)
- 3) mobile сценарий:
 - выбрать отель → номер
 - проверить доступность на диапазон
 - забронировать
 - показать, что пересечение запрещается
 - отменить бронь
- 4) web сценарий:
 - список
 - создать/отменить бронь
- 5) Windows-виджет:
 - вид статуса

- обновление
- детали (если сделали)

7) Что сдавать

В репозитории:

- исходники backend / mobile / windows / web
- `docker-compose.yml`
- `README.md` (как запустить и проверить)
- ссылка на видео (YouTube/Drive/другой хостинг)
- (опционально) краткое описание архитектурных решений и компромиссов

8) Критерии оценки

- ****Воспроизводимость:**** запускается по README, работает без ручной магии
- ****Работоспособность:**** все сценарии из раздела 1 покрыты
- ****Качество:**** структура кода, ясные названия, базовая валидация, обработка ошибок
- ****Стабильность:**** корректная логика непересечения броней
- ****Инженерность:**** разумные решения по данным, API, состоянию, UI
- ****Коммуникация:**** понятный README + внятное видео

9) Дополнительно (необязательно, но плюс)

- автотесты (backend и/или frontend)
- генерация типобезопасного клиента GraphQL
- подписки/обновления (subscriptions) или polling
- базовая аналитика/логирование событий
- CI (lint/test/build)